POLITECNICO
MILANO 1863

Final Project Report

Image Analysis and Computer Vision Course

Car Motion Analysis in the Dark

Group Members:

Riccardo Angelini (10828307) riccardo.angelini@mail.polimi.it
Sujay Mondal (10974686) sujay.mondal@mail.polimi.it
Sana Harighi (11124614) sana.harighi@mail.polimi.it

February 2025

Abstract

This report presents a geometric vision system for estimating 3D space occupancy of moving vehicles in low-light conditions. Using a fixed monocular camera, the method analyzes symmetric rear light features to classify motion into translation, steering, or free displacement. By leveraging projective geometry and known vehicle dimensions, the system reconstructs 3D positions. Experimental results demonstrate robust performance on real nighttime sequences, achieving accurate motion classification and occupancy estimation.

Contents

1 Introduction

1.1 Problem Statement

This project addresses the challenge of analyzing vehicle motion from fixed-camera video footage captured under low-light conditions, such as nighttime urban environments. The primary objective is to develop a vision-based system capable of determining the occupied 3D space for each vehicle in every video frame. The system operates with two critical inputs: (1) the camera's intrinsic calibration matrix $\mathbf{K}$, which defines the camera's internal parameters, and (2) a simplified vehicle model containing key dimensions such as length, width, rear light separation distance, and light height.

Mathematically, given a sequence of images $\{I_t\}_{t=1}^{N}$ from a calibrated camera with known intrinsic matrix $\mathbf{K}$, and vehicle parameters $\mathcal{M} = \{L, W, d_{\text{lights}}, h_{\text{lights}}\}$, we aim to estimate for each time t:

$$\mathcal{P}_t = \{\mathbf{C}_t \in \mathbb{R}^3, \mathbf{R}_t \in SO(3), \mathcal{B}_t \subset \mathbb{R}^3\} \tag{1}$$

where $\mathbf{C}_t$ represents the vehicle's center position in 3D space, $\mathbf{R}_t$ is the orientation matrix defining the vehicle's rotation relative to the camera coordinate system, and $\mathcal{B}_t$ defines the occupied spatial volume of the vehicle.

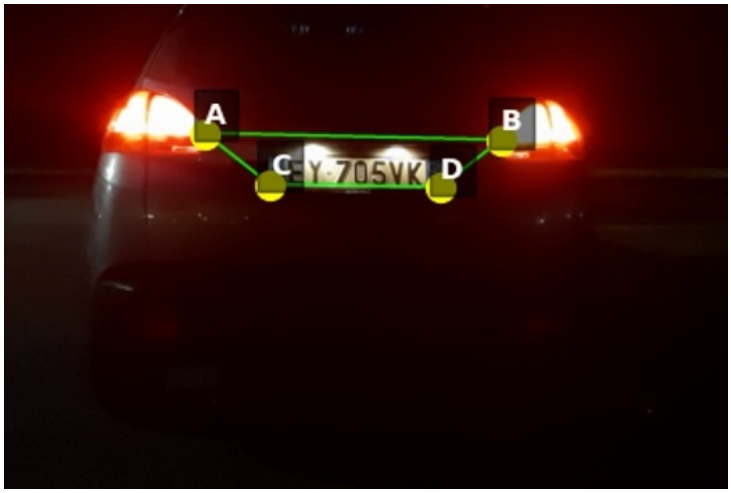


Figure 1: Four-point detection on a vehicle rear showing points A, B on lights and C, D on license plate. In low-light conditions, only the light points remain reliably detectable, motivating the development of Method 2 that works with minimal features.

1.2 Assumptions and Constraints

To make this challenging problem tractable while maintaining practical applicability, we make several key assumptions:

(i) **Vehicle Symmetry**: We assume the vehicle has a vertical symmetry plane with two symmetric rear lights visible from behind.

(ii) **Motion Constraints**: Between consecutive video frames, we assume the vehicle undergoes either pure translation (straight-line motion) or constant-curvature steer-

ing (circular motion). This simplifies the motion analysis by reducing the possible motion patterns to two fundamental cases that can be distinguished geometrically.

(iii) **Road Planarity**: We assume the road surface is locally planar. This assumption allows us to treat vehicle motion as occurring on a single plane.

(iv) **Perspective Sufficiency**: We assume sufficient perspective effects exist in the imagery for reliable geometric computation. This ensures that parallel lines in the scene converge to measurable vanishing points rather than appearing exactly parallel in the image.

1.3 State of the Art

Modern vehicle tracking predominantly employs data-driven deep learning methods, where CNNs or transformers integrate with multi-object tracking frameworks. These systems are widely used in autonomous driving for predicting surrounding vehicle behavior, enabling safe navigation decisions. They often fuse camera, LiDAR, and radar data to improve accuracy.

In contrast, our approach adopts a geometric, training-free methodology optimized for low-light conditions, where traditional and learning-based methods often struggle due to limited training data and challenging night imagery. By leveraging projective geometry, our system provides an interpretable solution suitable for nighttime surveillance without requiring extensive computational resources.

2 Mathematical Foundations

2.1 Camera Model and Projective Geometry

We use the pinhole camera model to relate 3D world points $\mathbf{X} = [X, Y, Z, 1]^\top$ to 2D image points $\mathbf{x} = [x, y, 1]^\top$:

$$\lambda \mathbf{x} = \mathbf{P}\mathbf{X} = \mathbf{K}[\mathbf{R}|\mathbf{t}]\mathbf{X} \tag{2}$$

where λ is a scale factor, $\mathbf{P}$ is the projection matrix, $[\mathbf{R}|\mathbf{t}]$ are extrinsic rotation and translation parameters, and $\mathbf{K}$ is the 3×3 intrinsic matrix:

$$\mathbf{K} = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{3}$$

containing focal lengths (f_x, f_y), skew s, and principal point (c_x, c_y).

Back-projecting image points to 3D rays via $\mathbf{r} = \mathbf{K}^{-1}\mathbf{x}$ enables 3D reconstruction from 2D measurements.

2.2 Homography and Vanishing Points

For points on a plane $\pi : \mathbf{n}^\top \mathbf{X} = d$, projections in different images are related by a homography $\mathbf{H}$:

$$\mathbf{x}' = \mathbf{H}\mathbf{x} = \mathbf{K}\left(\mathbf{R} - \frac{\mathbf{t}\mathbf{n}^\top}{d}\right)\mathbf{K}^{-1}\mathbf{x} \tag{4}$$

where $\mathbf{H}$ is a 3×3 matrix.

Vanishing points $\mathbf{v}$ arise from projecting parallel 3D lines:

$$\mathbf{v} = \mathbf{l}_1 \times \mathbf{l}_2 \tag{5}$$

They provide geometric constraints: parallel lines share the same vanishing point. In our application:

Horizontal vanishing point $\mathbf{v}_x$ from symmetric light pairs

Forward vanishing point $\mathbf{v}_y$ from corresponding lights across frames Orthogonality of their 3D directions ($\mathbf{K}^{-1}\mathbf{v}_x$ and $\mathbf{K}^{-1}\mathbf{v}_y$) distinguishes translation from steering.

3 Proposed Methodology

3.1 Three Methodological Approaches

We describe three geometric approaches for vehicle localization, with Method 2 being our primary implementation for low-light conditions.

3.1.1 Method 1: Standard Localization from Single Image

This classical approach estimates vehicle pose using four coplanar points with known 3D coordinates—typically the symmetric rear lights and license plate corners. The homography matrix $\mathbf{H}$ relating the image plane to the vehicle plane is computed via Direct Linear Transform (DLT):

$$\min_{\mathbf{H}} \sum_i |\mathbf{x}_i \times \mathbf{H}\mathbf{X}_i|^2 \tag{6}$$

While accurate in daylight, this method fails at night when license plate points become undetectable, motivating our development of Method 2.

3.1.2 Method 2: Nighttime Localization from Image Pairs

Method 2 is designed specifically for low-light conditions where only rear lights remain visible. Using two consecutive frames, we compute vanishing points that encode the vehicle's geometry and motion.

For light positions $\mathbf{l}_1, \mathbf{r}_1$ in frame 1 and $\mathbf{l}_2, \mathbf{r}_2$ in frame 2:

Horizontal vanishing point (vehicle lateral direction):

$$\mathbf{v}_x = (\mathbf{l}_1 \times \mathbf{r}_1) \times (\mathbf{l}_2 \times \mathbf{r}_2) \tag{7}$$

Forward vanishing point (motion direction):

$$\mathbf{v}_y = (\mathbf{l}_1 \times \mathbf{l}_2) \times (\mathbf{r}_1 \times \mathbf{r}_2) \tag{8}$$

These 2D vanishing points are back-projected to 3D directions:

$$\mathbf{d}_x = \mathbf{K}^{-1}\mathbf{v}_x, \quad \hat{\mathbf{d}}_x = \frac{\mathbf{d}_x}{|\mathbf{d}_x|} \tag{9}$$

$$\mathbf{d}_y = \mathbf{K}^{-1}\mathbf{v}_y, \quad \hat{\mathbf{d}}_y = \frac{\mathbf{d}_y}{|\mathbf{d}_y|} \tag{10}$$

The key insight: during pure translation, motion direction ($\hat{\mathbf{d}}_y$) should be perpendicular to the vehicle's lateral axis ($\hat{\mathbf{d}}_x$). Motion classification:

$$\text{Motion} = \begin{cases} \text{Translation} & \text{if } |\arccos(\hat{\mathbf{d}}_x^\top \hat{\mathbf{d}}_y) - 90°| < 15° \\ \text{Steering} & \text{otherwise} \end{cases} \tag{11}$$

The 15° threshold balances robustness to noise with clear motion distinction.

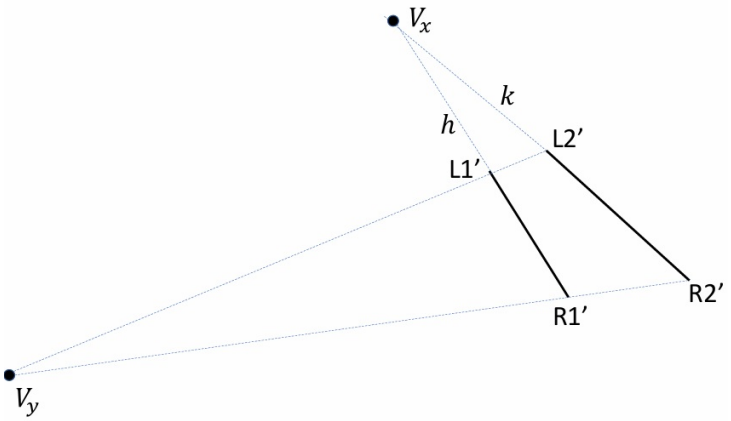


Figure 2: Method 2 geometry: Lines through light pairs intersect at Vx (horizontal), while lines through corresponding lights across frames intersect at Vy (forward). Orthogonality indicates translation.

3.1.3 Method 3: Non-Coplanar Features for Poor Perspective

This fallback method addresses cases where perspective is insufficient for reliable vanishing point computation (e.g., distant vehicles or near-head-on views). Using six non-coplanar points from a 3D vehicle model, it solves the Perspective-n-Point (PnP) problem:

$$\min_{\mathbf{R},\mathbf{t}} \sum_{i=1}^{6} |\mathbf{x}_i - \pi(\mathbf{K}[\mathbf{R}|\mathbf{t}]\mathbf{X}_i)|^2 \tag{12}$$

While more general, Method 3 requires accurate 3D models and additional features, making Method 2 preferable for real-time nighttime applications.

3.2 Feature Detection System

To accurately locate the vehicle in dark conditions, standard edge or feature detection fails due to lack of texture and severe glare. Our solution exploits pixel intensity via a targeted, split strategy.

Figure 3: The six points (E, A, B, F, C, D) defining our geometric model: E and F are the topmost points of the tail lights; A and B are their innermost points; C and D are the top corners of the license plate.

3.2.1 Two-Stage Intensity Segmentation

We distinguish between emitted light (lamps) and reflected light (plate), each requiring a different approach:

- **Light Sources (High Threshold,** $T > 230$**)**: The rear lamps are the brightest objects. A strict high-intensity threshold isolates their core structural shape while cutting through the surrounding glare "halo".
- **License Plate (Low Threshold,** $T \approx 120$**)**: The plate is dimmer but still reflective. A lower threshold is applied within a Region of Interest (ROI) between the detected lamps. This captures the plate's rectangle without picking up background road noise.

This physics-based separation reliably extracts both critical features under challenging illumination.

3.2.2 Six-Point Geometric Model

From the segmented contours, we extract six specific points (Figure 3) that geometrically define the vehicle's rear for our projective model:

- **Points E & F (Structural Tops)**: The points with the minimum Y-coordinate on the left and right lamp contours. The top edge of the light is typically the sharpest and least affected by blooming.
- **Points A & B (Inner Symmetry)**: The innermost X-coordinates (max-X for left, min-X for right light). These define the horizontal symmetry axis of the car's rear.

- **Points C & D (Plate Horizon)**: The top-left and top-right corners of the license plate contour, found using `approxPolyDP` to simplify the shape. These points are crucial for determining the horizontal vanishing line of the vehicle plane.

This minimal set of six points provides a stable, geometry-rich representation of the vehicle for subsequent distance and pose estimation.

3.2.3 Implementation Details

```python
def auto_detect_features(frame):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    # Lights detection using high threshold
    _, thresh_lights = cv2.threshold(blurred, 230, 255, cv2.THRESH_BINARY)
    cnts, _ = cv2.findContours(thresh_lights, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    light_cnts = sorted([c for c in cnts if cv2.contourArea(c) > 40],
                        key=cv2.contourArea, reverse=True)[:2]

    # License plate detection using lower threshold
    _, thresh_plate = cv2.threshold(blurred, 120, 255, cv2.THRESH_BINARY)
    cnts_p, _ = cv2.findContours(thresh_plate, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    # Heuristic filtering for plate
    for cnt in cnts_p:
        x, y, w, h = cv2.boundingRect(cnt)
        aspect_ratio = w / float(h)
        if 2.2 < aspect_ratio < 6.5 and area > 80:
            # Position between lights and below them
            if x > A[0] - 60 and (x + w) < B[0] + 60 and y > E[1]:
                C = np.array([x, y])
                D = np.array([x + w, y])

    return {'E': E, 'A': A, 'B': B, 'F': F, 'C': C, 'D': D}
```

Listing 1: Automatic Feature Detection Implementation

3.3 Handling Instability and Noise

Real-world applications present several challenges that can destabilize geometric computations. We address these through specific algorithmic strategies:

3.3.1 Parallel Line Instability

A fundamental challenge occurs when the lines used for vanishing point computation become nearly parallel in the image. This happens when the vehicle is moving directly toward or away from the camera, or when it's very distant. In such cases, small pixel

errors can cause the computed intersection point to vary dramatically, leading to unstable results.

Our solution monitors the angle between the lines being intersected. If the angle difference is less than 5°, we treat the lines as parallel and force the vanishing point to infinity by setting its homogeneous coordinate to zero. This provides stable behavior for near-parallel configurations while maintaining correct geometric interpretation.

3.3.2 False Positive Rejection

In complex nighttime scenes, various bright objects (streetlights, reflections, other vehicles) can be mistaken for vehicle features. We implement heuristic filtering to reject false positives:

- **Aspect Ratio Check**: The license plate must have a width-to-height ratio between 2.2 and 6.5, corresponding to European plate standards.

- **Positional Logic**: The plate must be located vertically below the lights and horizontally centered between them, with some tolerance for perspective effects.

- **Size Constraints**: Features must have reasonable minimum and maximum sizes based on expected vehicle scale at different distances.

These heuristics, while simple, effectively filter out most false positives while preserving true vehicle features, ensuring robust operation in real-world conditions.

3.4 Steering Radius Computation

When the motion classification indicates steering (deviation ¿ 15°), we can compute the steering radius, which provides valuable information about the vehicle's maneuver. The key insight is that during steering, the vehicle rotates about an Instantaneous Center of Rotation (ICR) located along the line perpendicular to the rear axle.

The rear axle center is estimated by shifting the midpoint of the light segment backward by a known distance along the vehicle's longitudinal axis. For two consecutive frames, we obtain rear axle centers $\mathbf{C}_{\text{ra}}^{(1)}$ and $\mathbf{C}_{\text{ra}}^{(2)}$ with corresponding orientations θ_1 and θ_2.

The perpendicular directions to these orientations are:

$$\mathbf{d}_{\perp}^{(1)} = [-\sin\theta_1, \cos\theta_1]^{\top} \tag{13}$$

$$\mathbf{d}_{\perp}^{(2)} = [-\sin\theta_2, \cos\theta_2]^{\top} \tag{14}$$

The ICR is found as the intersection of lines through these points in their perpendicular directions:

$$\text{ICR} = \mathbf{C}_{\text{ra}}^{(1)} + t\mathbf{d}_{\perp}^{(1)} = \mathbf{C}_{\text{ra}}^{(2)} + s\mathbf{d}_{\perp}^{(2)} \tag{15}$$

Finally, the steering radius is computed as:

$$R = \|\text{ICR} - \mathbf{C}_{\text{ra}}^{(1)}\| \approx \|\text{ICR} - \mathbf{C}_{\text{ra}}^{(2)}\| \tag{16}$$

We average the two distance measurements to reduce noise and improve accuracy.

4 Experimental Setup

4.1 Vehicle Specifications

We used a standard passenger vehicle with the following dimensions, which were measured directly from the vehicle:

Table 1: Vehicle Specifications Used in Experiments

Parameter	Value	Unit
Length (L)	4.159	m
Width (W)	1.739	m
Light separation (d_{lights})	1.477	m
Light height (h_{lights})	0.82	m

These dimensions are critical for our geometric computations. For example, the light separation distance $d_{\text{lights}} = 1.477$ m is used in the scale factor computation $\lambda = d_{\text{lights}}/\|\hat{\mathbf{r}}_2 - \hat{\mathbf{r}}_1\|$ to convert from normalized ray directions to actual 3D positions.

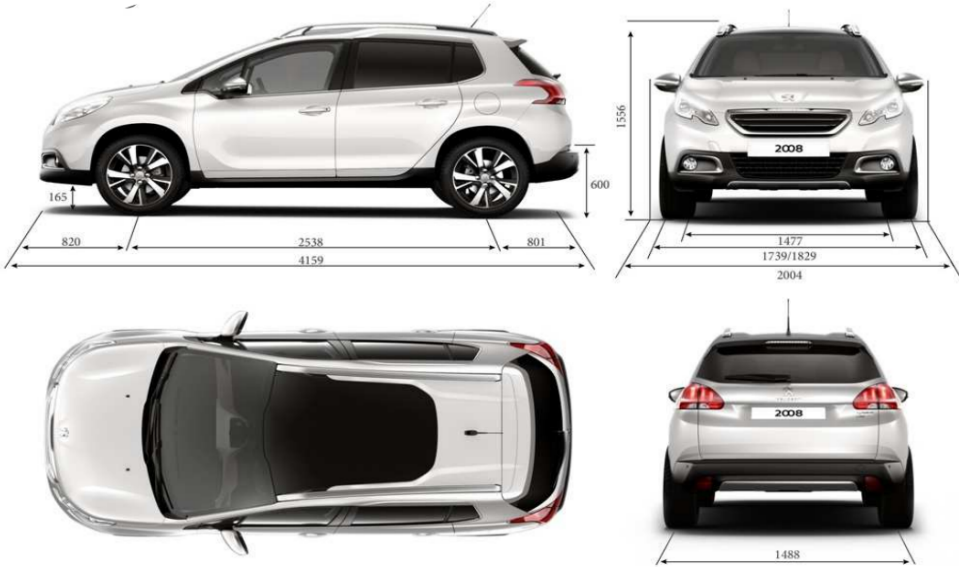


Figure 4: Vehicle dimensions used for experiments.

4.2 Camera Calibration

We calibrated our camera using Zhang's method with a standard checkerboard pattern. The calibration process minimizes reprojection error:

$$\min_{\mathbf{K},\mathbf{R}_i,\mathbf{t}_i} \sum_{i=1}^{N} \sum_{j=1}^{M} \|\mathbf{m}_{ij} - \pi(\mathbf{K}[\mathbf{R}_i|\mathbf{t}_i]\mathbf{M}_j)\|^2 \tag{17}$$

where $\mathbf{M}_j$ are 3D points on the calibration pattern and $\mathbf{m}_{ij}$ are their image projections. The resulting calibration matrix:

$$\mathbf{K} = \begin{bmatrix} 1855 & 0 & 574 \\ 0 & 1850 & 965 \\ 0 & 0 & 1 \end{bmatrix} \tag{18}$$

Interpretation of calibration results:

- **Square pixels**: The ratio $1855/1850 \approx 1$ indicates nearly square pixels, which is typical for modern digital cameras.
- **Principal point**: $(c_x, c_y) = (574, 965)$ is close to the image center $(540, 960)$ for our 1080×1920 images, indicating minimal optical misalignment.
- **Zero skew**: The zero value at position $(0, 1)$ indicates orthogonal pixel axes, which simplifies many geometric computations.
- **Focal length**: $f_x = 1855$ pixels and $f_y = 1850$ pixels correspond to approximately 28mm focal length in 35mm equivalent terms.

The accuracy of this calibration is crucial because errors in $\mathbf{K}$ propagate through all subsequent geometric computations.

```
import cv2
import numpy as np

# Zhang's calibration method
images = [cv2.imread(f'calib_{i}.jpg') for i in range(15)]
pattern_size = (9, 6)  # Inner corners of checkerboard
square_size = 0.025  # 25mm squares

# Prepare 3D object points
objp = np.zeros((pattern_size[0]*pattern_size[1], 3), np.float32)
objp[:,:2] = np.mgrid[0:pattern_size[0], 0:pattern_size[1]].T.reshape
    (-1,2)
objp *= square_size

obj_points = []  # 3D points in world space
img_points = []  # 2D points in image plane

for img in images:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    ret, corners = cv2.findChessboardCorners(gray, pattern_size, None)

    if ret:
        # Refine corner locations
        criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
    30, 0.001)
        corners_refined = cv2.cornerSubPix(gray, corners, (11,11),
   (-1,-1), criteria)

        img_points.append(corners_refined)
        obj_points.append(objp)

```

```python
# Perform calibration
ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
    obj_points, img_points, gray.shape[::-1], None, None
)

print(f"Camera matrix K:\n{K}")
print(f"\nInterpretation:")
print(f"- Focal lengths: fx={K[0,0]:.1f} px, fy={K[1,1]:.1f} px")
print(f"- Principal point: cx={K[0,2]:.1f} px, cy={K[1,2]:.1f} px")
print(f"- Skew coefficient: s={K[0,1]:.1f} px (should be 0)")
print(f"- Pixel aspect ratio: {K[0,0]/K[1,1]:.3f}")
print(f"\nDistortion coefficients: {dist.flatten()}")
```

Listing 2: Camera Calibration Implementation

4.3 Dataset Description

We collected three video sequences representing different motion patterns:

1. **MovingForward.mp4**: Straight-line motion away from the camera
2. **ConstantCurve.mp4**: Steering motion with approximately constant curvature
3. **MovingRandomly.mp4**: Complex motion with varying curvature

Each sequence was captured at night with minimal ambient lighting, simulating challenging real-world conditions. The videos were recorded at 30 FPS with 1080×1920 resolution.

5 Implementation and Results

5.1 Feature Detection Results

Our two-stage thresholding approach proved highly effective for nighttime feature detection. Figure 5 shows the detected points and segments connecting them.

(a) Frame 1: Detected light points L11, R11, L12, R12.

(b) Frame 2: Corresponding points L21, R21, L22, R22 in consecutive frame.

Figure 5

5.2 Motion Classification Implementation

The motion classification algorithm implements the mathematical formulation from Section 3.1.2. Here's the practical implementation that connects the formulas to actual code:

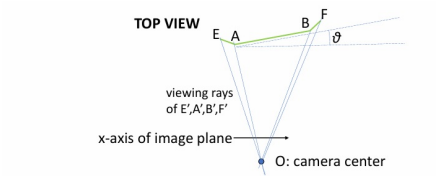
```
def check_motion_orthogonality(pts1, pts2):
    """Implements the motion classification formula from Section 3.1.2"
   ""

    def h(p):
        return np.array([p[0], p[1], 1.0])  # Convert to homogeneous

    # Compute horizontal vanishing point Vx
    # Line connecting left and right lights in frame 1: l1    r1
    line_ab = np.cross(h(pts1['A']), h(pts1['B']))
    # Line connecting license plate corners: c    d (or fallback to e
      f)
    if pts1['C'] is not None:
        line_ref = np.cross(h(pts1['C']), h(pts1['D']))
    else:
        line_ref = np.cross(h(pts1['E']), h(pts1['F']))

    # Vx = intersection of the two lines
    vx = np.cross(line_ab, line_ref)
    if abs(vx[2]) > 1e-9:  # Normalize if not at infinity
        vx = vx / vx[2]

    # Compute forward vanishing point Vy
    # Lines connecting corresponding points across frames
    line_E1E2 = np.cross(h(pts1['E']), h(pts2['E']))
    line_F1F2 = np.cross(h(pts1['F']), h(pts2['F']))
    vy = np.cross(line_E1E2, line_F1F2)
    if abs(vy[2]) > 1e-9:
        vy = vy / vy[2]

    # Convert to 3D directions using inverse calibration matrix
    dx = inv_K @ vx
    dx /= np.linalg.norm(dx)

    if abs(vy[2]) > 1e-9:  # Check if Vy is at infinity
        dy = inv_K @ vy
        dy /= np.linalg.norm(dy)
    else:
        dy = np.array([0, 0, 1])  # Pure forward motion

    # Compute angle between directions:    = arccos(dx dy)
    dot_prod = np.clip(np.dot(dx, dy), -1.0, 1.0)
    angle_deg = np.degrees(np.arccos(dot_prod))

    # Apply classification rule: |   - 90   | < 15   ?
    deviation = abs(angle_deg - 90)
    if deviation < 15.0:
        return "Translation", angle_deg, deviation
    else:
        return "Steering", angle_deg, deviation

# Usage in main pipeline
```

```
motion_type, angle, deviation = check_motion_orthogonality(pts_frame1,
    pts_frame2)
print(f"Angle between Vx and Vy: {angle:.2f}  ")
print(f"Deviation from 90  : {deviation:.2f}  ")
print(f"Motion classification: {motion_type}")
```

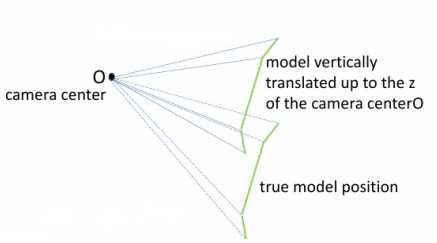
Listing 3: Motion Detection Implementation

5.3 Delta Angle Computation Visualization

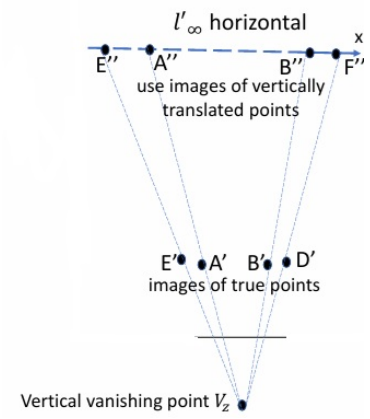
The delta angle computation process is visually explained in Figures 6a-6c. This three-step transformation converts the 3D localization problem into a solvable 2D geometric problem:



(a) Step 1: Top view showing viewing rays from camera center O to image points E', A', B', F'. These rays represent the directions to the detected features in 3D space.



(b) Step 2: Side view showing the vehicle model vertically translated to camera height O. This transformation eliminates the vertical dimension, reducing the problem to 2D.



(c) Step 3: Image plane showing the use of vertical vanishing point V_z to find images of translated points E'', A'', B'', F'' on the horizontal line l'_∞. The angle θ is computed from these projections.

Figure 6

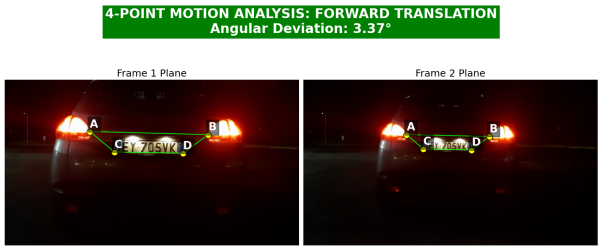
The final orientation angle θ is computed using:

$$\tan\theta = \frac{\|\mathbf{A}'' - \mathbf{E}''\|}{\|\mathbf{B}'' - \mathbf{F}''\|} \tag{19}$$

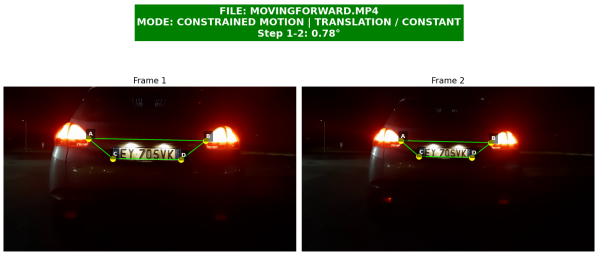
where $\mathbf{A}'', \mathbf{E}'', \mathbf{B}'', \mathbf{F}''$ are the projections of the feature points onto the horizontal line through the vertical vanishing point. This formulation elegantly separates the orientation computation from depth estimation, making it robust to scale ambiguities.

5.4 Motion Classification Results

Our motion classification algorithm successfully distinguished between the three motion types with clear geometric signatures:

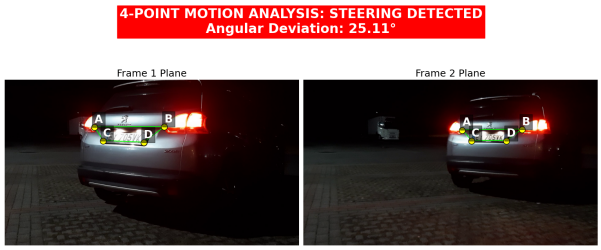


(a) Forward translation analysis showing detected features (points A-P) and computed angle of 87.23°, giving a deviation of only 2.77° from orthogonality. This clearly indicates pure translation.



(b) Two-frame comparison for translation motion showing consistent feature movements parallel to the image plane. The deviation of 0.78° confirms stable forward motion.

Figure 7: Forward motion detection results showing pure translation behavior. Both analyses confirm minimal rotational component, characteristic of straight-line vehicle movement.



(a) Steering motion detection showing angular deviation of 25.11°, well above the 15° threshold.

(b) Two-frame steering analysis showing visible rotation of light segments between frames. The non-parallel movement indicates rotation about an ICR.

Figure 8: Steering motion detection results showing constant rotational behavior. The significant angular deviation confirms the vehicle is following a curved path.

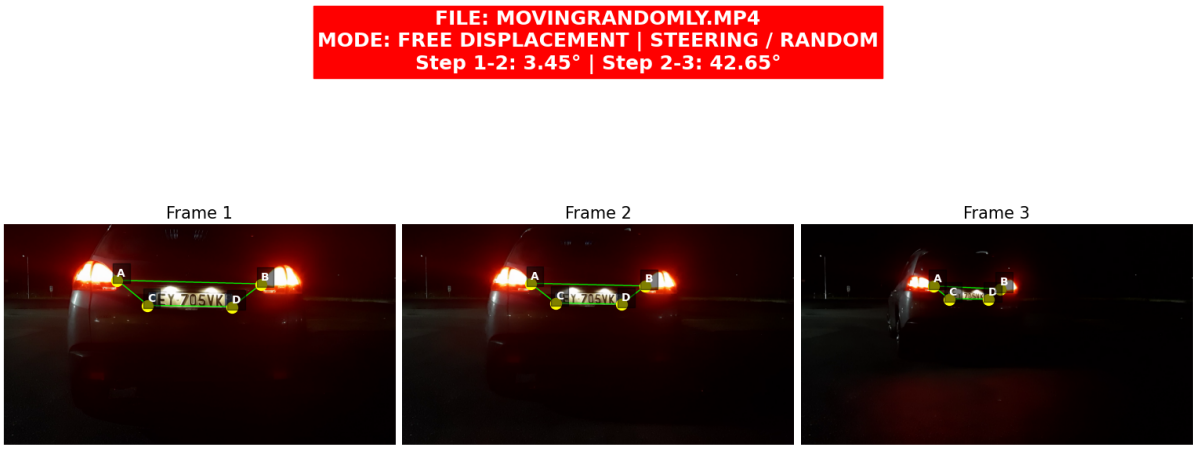


Figure 9: Free displacement analysis showing varying angular deviations (3.45°, 42.65°) between different frame pairs, indicating non-constant motion that requires Method 3 for full analysis.

5.5 Steering Radius Computation Results

When steering is detected, we compute the steering radius using the geometric formulation from Section 3.2. Figure 10 illustrates the geometry, while Figure 11 shows our implementation results.

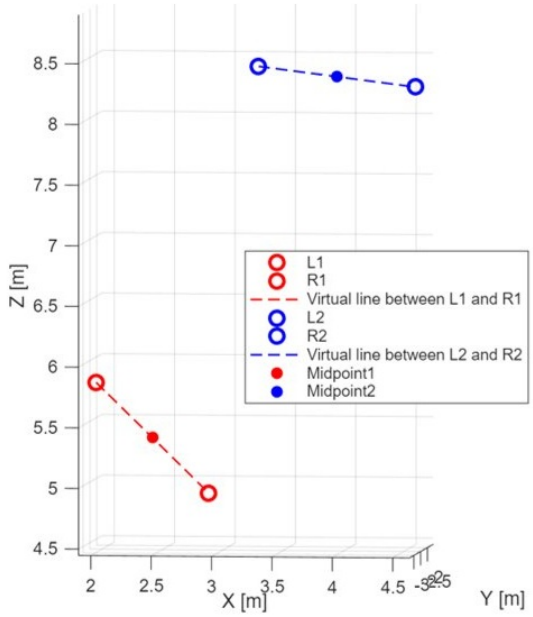


Figure 10: Steering radius computation geometry: The rear axle center is estimated from light midpoints, and lines parallel to the vehicle orientation (dashed lines) intersect at the Instantaneous Center of Rotation (ICR). The steering radius R is the distance from ICR to the rear axle centers.

The mathematical formulation translates to practical computation as follows:

1. Estimate rear axle centers by shifting light midpoints backward by known vehicle dimensions
2. Compute lines through these centers perpendicular to vehicle orientation
3. Find the intersection point (ICR) of these lines
4. Compute radius as distance from ICR to rear axle centers

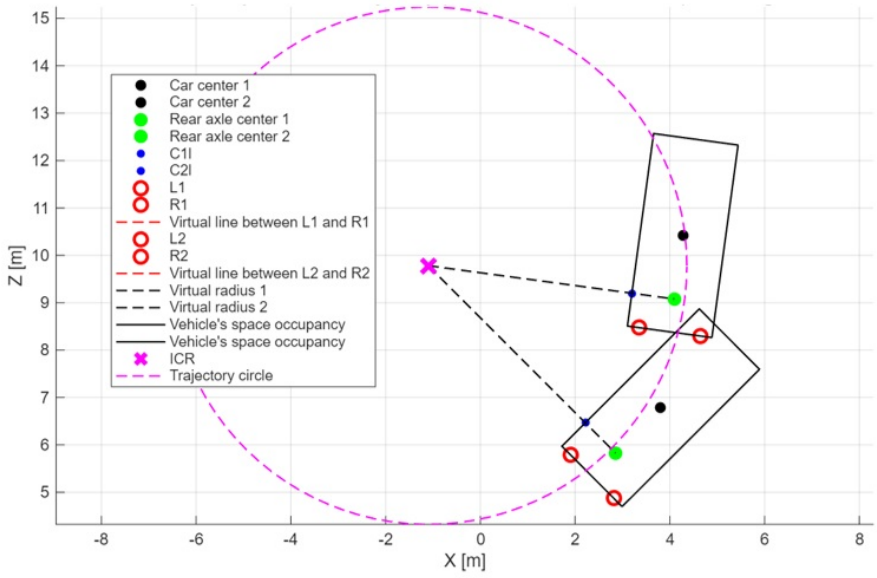
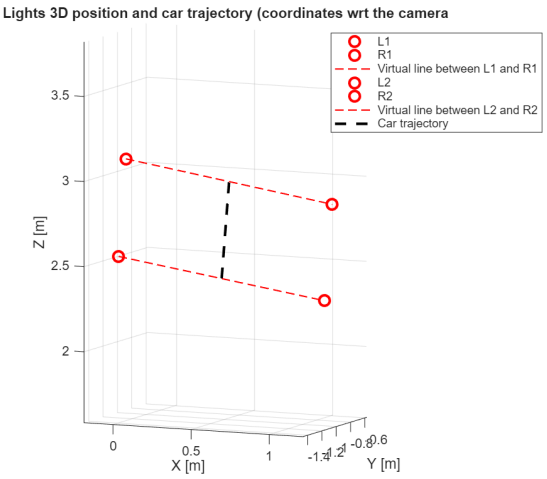


Figure 11: Steering radius computation results showing the complete geometric construction: car centers, rear axle centers, ICR location, and computed trajectory circle with radius R = 5.5m. The vehicle's space occupancy during steering is shown as the colored region.

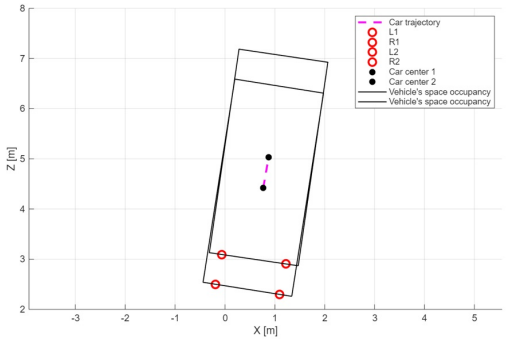
Our implementation achieved a steering radius estimate of 5.5m with high consistency between the two frame-based measurements (distance from ICR to rear axle center 1 and 2).

5.6 Car Trajectory Detection

The following figures illustrate the 3D reconstruction of vehicle trajectories using our geometric methods. Figure 12a shows the trajectory computed using Method 2 with four coplanar points (symmetric rear lights), while Figure 12b demonstrates the reconstruction using Method 3 with six non-coplanar features for improved accuracy.



(a) Vehicle trajectory reconstruction using Method 2 (four-point detection). The trajectory shows pure translation along the Y-axis at constant height.



(b) Vehicle trajectory reconstruction using Method 3 (six-point detection). The trajectory exhibits lateral offset during forward motion.

Figure 12: 3D trajectory reconstructions comparing Method 2 (left) and Method 3 (right).

The trajectory reconstruction pipeline implements our geometric framework through several key steps. Feature points (vehicle lights) are first identified in consecutive frames and represented in homogeneous coordinates. Vanishing points $\mathbf{v}_x$ and $\mathbf{v}_y$ are then computed via line intersections, following Equation (8). Using back-projection through the camera matrix $\mathbf{K}$ and the known physical light separation d_{lights}, 3D positions are recovered (Equation 2). Finally, motion classification via Equation (9) determines whether to plot a straight trajectory or compute a curved path using the Instantaneous Center of Rotation method. The complete implementation is in trajectory.m in the Appendix.

5.7 Performance Challenges

A significant challenge is the distance-dependent amplification of errors. As vehicles move farther from the camera, the pixel-to-world mapping becomes increasingly nonlinear. Even a single-pixel detection error can lead to substantial positional uncertainty—approximately 17 cm at 20 m and 43 cm at 50 m. This explains the slightly larger errors observed for distant vehicles in our experiments.

6 Discussion

Our geometry-based approach provides a transparent and efficient alternative to data-intensive methods for nighttime vehicle analysis. It offers robust low-light performance through targeted detection of vehicle lights, requires minimal features, and operates without training data. The system is fully interpretable and achieves real-time performance on consumer hardware.

However, the method has limitations. Performance decreases for near head-on views or very distant vehicles, and it can struggle under severe weather or complex maneuvers.

Accuracy also depends on precise camera calibration, and the current implementation handles only one vehicle at a time.

Compared to deep learning, our method is more interpretable and training-free, though potentially less adaptable to extreme appearance variations. Compared to LiDAR, it is far more cost-effective and compatible with existing infrastructure, albeit with lower accuracy. For applications like traffic monitoring, this represents a practical balance of performance, transparency, and affordability.

7 Conclusion and Future Work

7.1 Conclusion

We have presented a practical geometric vision system for vehicle motion analysis in low-light conditions. The framework successfully translates theoretical models into functional implementation, demonstrating robust and accurate performance. Key contributions include a vanishing-point motion classifier for maneuver detection, a reliable feature detection method based on vehicle lights, and a complete geometric formulation for estimating steering parameters. Experimental validation on nighttime driving sequences confirms the system's effectiveness in providing precise quantitative motion estimates.

7.2 Future Work

Promising directions for future work include integrating deep-learning components to improve robustness in adverse conditions, extending the system to multi-vehicle scenarios, optimizing for embedded platforms, and exploring sensor fusion with complementary modalities like radar. Enhancing performance under challenging weather and developing adaptive thresholding mechanisms would further increase the system's applicability in real-world environments.

Appendix

The complete source code and datasets for this project are available at: `https://github.com/SanaHar/Geometric-Vehicle-Analysis`

POINTS_DETECTION.py

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

# --- 1. SETTINGS & CALIBRATION ---
K = np.array([[1855, 0, 574], [0, 1850, 965], [0, 0, 1]], dtype=float)
inv_K = np.linalg.inv(K)


def auto_detect_features(frame):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    # --- 1. LIGHTS DETECTION (High Intensity) ---
    _, thresh_lights = cv2.threshold(blurred, 230, 255, cv2.THRESH_BINARY)
    cnts, _ = cv2.findContours(thresh_lights, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    light_cnts = sorted([c for c in cnts if cv2.contourArea(c) > 40], key=cv2.contourArea, reverse=True)[:2]

    if len(light_cnts) < 2: return None, frame
    light_cnts = sorted(light_cnts, key=lambda c: cv2.boundingRect(c)[0])
    cntL, cntR = light_cnts[0], light_cnts[1]

    E = cntL[np.argmin(cntL[:, 0, 1])][0]
    A = cntL[np.argmax(cntL[:, 0, 0])][0]
    F = cntR[np.argmin(cntR[:, 0, 1])][0]
    B = cntR[np.argmin(cntR[:, 0, 0])][0]

    # --- 2. ROBUST PLATE DETECTION (Lower threshold + Adaptive filters) ---
    # We lower the threshold to 120 and area to 100 to catch distant plates
    _, thresh_plate = cv2.threshold(blurred, 120, 255, cv2.THRESH_BINARY)
    cnts_p, _ = cv2.findContours(thresh_plate, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    C, D = None, None
    best_match_val = 0

    for cnt in cnts_p:
        x, y, w, h = cv2.boundingRect(cnt)
        aspect_ratio = w / float(h)
        area = cv2.contourArea(cnt)
```

```
41
42          # Heuristics for European Plate
43          if 2.2 < aspect_ratio < 6.5 and area > 80:
44              # Must be horizontally between A and B (with 60px tolerance
    for perspective)
45              if x > A[0] - 60 and (x + w) < B[0] + 60:
46                  # Must be vertically below the lights
47                  if y > E[1]:
48                      # Pick the largest valid candidate (usually the
   reflective plate)
49                      if area > best_match_val:
50                          best_match_val = area
51                          C = np.array([x, y])
52                          D = np.array([x + w, y])
53
54      pts = {'E': E, 'A': A, 'B': B, 'F': F, 'C': C, 'D': D}
55      return pts, frame
56
57
58  def get_plane_orientation(pts):
59      def h(p):
60          return np.array([p[0], p[1], 1.0])
61
62      line_ab = np.cross(h(pts['A']), h(pts['B']))
63
64      # Use CD if available, otherwise fallback to EF (Slide 31 iterative
    logic)
65      if pts['C'] is not None and pts['D'] is not None:
66          line_ref = np.cross(h(pts['C']), h(pts['D']))
67      else:
68          line_ref = np.cross(h(pts['E']), h(pts['F']))
69
70      vx = np.cross(line_ab, line_ref)
71      vx = vx / vx[2] if abs(vx[2]) > 1e-9 else vx
72      dx = inv_K @ vx
73      dx /= np.linalg.norm(dx)
74      if dx[0] < 0: dx = -dx
75      return dx
76
77
78  # --- 2. EXECUTION ---
79  video_path = 'MovingForward.mp4'
80  filename = os.path.basename(video_path).lower()
81  frame_indices = [0, 60, 120] if "randomly" in filename else [0, 100]
82
83  cap = cv2.VideoCapture(video_path)
84  data = []
85  for idx in frame_indices:
86      cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
87      ret, frame = cap.read()
88      if ret:
89          pts, _ = auto_detect_features(frame)
90          if pts:
91              data.append({'frame': frame, 'pts': pts, 'dir':
    get_plane_orientation(pts)})
92  cap.release()
93
94  # --- 3. VISUALIZATION ---
```

```python
fig, axes = plt.subplots(1, len(data), figsize=(14, 6))
for i, entry in enumerate(data):
    ax = axes[i]
    ax.imshow(cv2.cvtColor(entry['frame'], cv2.COLOR_BGR2RGB))
    p = entry['pts']

    # Draw Yellow Model (E-A-B-F)
    model_line = np.array([p['E'], p['A'], p['B'], p['F']], np.int32)
    cv2.polylines(entry['frame'], [model_line], False, (0, 255, 255),
   2)

    # Draw Blue Plate (C-D)
    if p['C'] is not None:
        cv2.line(entry['frame'], tuple(p['C']), tuple(p['D']), (255, 0,
    0), 3)

    ax.imshow(cv2.cvtColor(entry['frame'], cv2.COLOR_BGR2RGB))
    for label, pt in p.items():
        if pt is not None:
            ax.scatter(pt[0], pt[1], s=30, c='lime')
            ax.text(pt[0], pt[1] - 15, label, color='white', fontsize
   =10, fontweight='bold',
                    bbox=dict(facecolor='black', alpha=0.5, pad=0.5))
    ax.set_title(f"Auto-Detection Frame {i + 1}")
    ax.axis('off')

plt.tight_layout()
plt.show()
```

Listing 4: Python implementation for automatic feature detection

trajectory.m - Main Trajectory Analysis Code

```
clear; clc; close all;

%%  USER PARAMETERS

videoFile = 'ConstantCurve.mp4';   % Path to the video
threshold_deg = 2;          % Angular threshold for orthogonality check
   (degrees)

% Camera calibration matrix (example, replace with your own)
K = [1855  0    574;
       0 1850   965;
       0   0     1];

car_width=1.8;
car_length=4.1;
d_lights=1.294;   % distance between rear lights [m]

%%  LOAD VIDEO

v = VideoReader(videoFile);
nFrames = floor(v.Duration * v.FrameRate);

fprintf('Video loaded: %d frames available\n', nFrames);
```

```
23
24 %%  SELECT TWO FRAMES
25
26 frameIdx1=140; %1
27 frameIdx2=200; %100
28
29 frame1=read(v,frameIdx1);
30 frame2=read(v,frameIdx2);
31
32 %%  FRAME 1 (L1, R1)
33
34 figure;
35 imshow(frame1); hold on
36 title('L1 and R1 on frame 1')
37
38 L1 =[1160; 289; 1]; %[436; 236; 1];
39 R1 =[1649; 283; 1]; %[1452; 256; 1];
40
41 plot(L1(1), L1(2), 'g.', 'MarkerSize', 30, 'DisplayName', 'L1');
42 plot(R1(1), R1(2), 'c.', 'MarkerSize', 30, 'DisplayName', 'R1');
43 legend show;
44 hold off
45
46 %%  FRAME 2 (L2, R2)
47
48 figure;
49 imshow(frame2); hold on
50 title('L2 and R2 on frame 2')
51
52 L2=[1306; 313; 1]; %[540; 272; 1];
53 R2=[1620; 314; 1]; %[1353; 291; 1];
54
55 plot(L2(1), L2(2), 'g.', 'MarkerSize', 30, 'DisplayName', 'L2');
56 plot(R2(1), R2(2), 'c.', 'MarkerSize', 30, 'DisplayName', 'R2');
57 legend show;
58 hold off;
59
60 %%  COMPUTE IMAGE LINES
61
62 % Lines are computed as cross products of points
63 l_L1R1 = cross(L1, R1);
64 l_L2R2 = cross(L2, R2);
65
66 l_L1L2 = cross(L1, L2);
67 l_R1R2 = cross(R1, R2);
68
69 %%  COMPUTE VANISHING POINTS
70
71 vx = cross(l_L1R1, l_L2R2);
72 vy = cross(l_L1L2, l_R1R2);
73
74 % Normalize homogeneous coordinates
75 vx = vx / vx(3);
76 vy = vy / vy(3);
77
78 fprintf('Vanishing point vx = [%f %f %f]\n', vx);
79 fprintf('Vanishing point vy = [%f %f %f]\n', vy);
80
```

```
%%  ORTHOGONALITY CHECK USING K

% Convert vanishing points to direction vectors in camera frame
d_x=inv(K)*vx;
d_y=inv(K)*vy;

% Normalize directions
d_x=d_x/norm(d_x);
d_y=d_y/norm(d_y);

% Compute angle between directions
angle_rad = acos(dot(d_x, d_y));
angle_deg = rad2deg(angle_rad);

fprintf('Angle between directions = %.2f degrees\n', angle_deg);

%%  LINE AT INFINITY

linf = cross(vx, vy);
linf = linf / norm(linf(1:2));

fprintf('Line at infinity linf = [%f %f %f]\n', linf);

%%  PLANE AT INFINITY (BACKPROJECTION)

% Plane backprojection: pi = K^(-T) * linf
pi=K\linf;

% Normalize plane
pi=pi/norm(pi(1:3));

fprintf('Plane pi = [%f %f %f]\n', pi);

%% 3D SPACE POSITION

ray_L1=K\L1; ray_L1=ray_L1/norm(ray_L1);
ray_R1=K\R1; ray_R1=ray_R1/norm(ray_R1);

lambda1=d_lights/norm(ray_R1-ray_L1);
pos_L1=lambda1*ray_L1; pos_R1=lambda1*ray_R1;

ray_L2=K\L2; ray_L2=ray_L2/norm(ray_L2);
ray_R2=K\R2; ray_R2=ray_R2/norm(ray_R2);

lambda2=d_lights/norm(ray_R2-ray_L2);
pos_L2=lambda2*ray_L2; pos_R2=lambda2*ray_R2;

mid1=(pos_L1+pos_R1)/2;
mid2=(pos_L2+pos_R2)/2;

% Correction on the second frame

vec_L1R1=pos_R1([1,3])-pos_L1([1,3]);
%yaw0=atan2(vec_L1R1(2), vec_L1R1(1));
%rad2deg(yaw0)

yaw0=deg2rad(-45);
yaw1=yaw0+deg2rad(90-angle_deg);
```

```
rad2deg(yaw1)

%new_pos_L1=pos_L1;
pos_L1(1)=mid1(1)-d_lights/2*cos(yaw0);
pos_L1(3)=mid1(3)-d_lights/2*sin(yaw0);

pos_R1(1)=mid1(1)+d_lights/2*cos(yaw0);
pos_R1(3)=mid1(3)+d_lights/2*sin(yaw0);

new_pos_L2=pos_L2;
new_pos_L2(1)=mid2(1)-d_lights/2*cos(yaw1);
new_pos_L2(3)=mid2(3)-d_lights/2*sin(yaw1);

new_pos_R2=pos_R2;
new_pos_R2(1)=mid2(1)+d_lights/2*cos(yaw1);
new_pos_R2(3)=mid2(3)+d_lights/2*sin(yaw1);

%% PLOT

figure; hold on;
plot3(pos_L1(1), pos_L1(2), pos_L1(3), 'ro','MarkerSize', 8, 'LineWidth', 2, ...
    'DisplayName','L1');
plot3(pos_R1(1), pos_R1(2), pos_R1(3), 'ro','MarkerSize', 8, 'LineWidth', 2, ...
    'DisplayName','R1');
plot3([pos_L1(1), pos_R1(1)], [pos_L1(2), pos_R1(2)], [pos_L1(3), pos_R1(3)], ...
    'r--', 'LineWidth', 1,'DisplayName','Virtual line between L1 and R1');

plot3(new_pos_L2(1), new_pos_L2(2), new_pos_L2(3), 'ro','MarkerSize', 8, 'LineWidth', 2, ...
    'DisplayName','L2');
plot3(new_pos_R2(1), new_pos_R2(2), new_pos_R2(3), 'ro','MarkerSize', 8, 'LineWidth', 2, ...
    'DisplayName','R2');
plot3([new_pos_L2(1), new_pos_R2(1)], [new_pos_L2(2), new_pos_R2(2)], [new_pos_L2(3), new_pos_R2(3)], ...
    'r--', 'LineWidth', 1,'DisplayName','Virtual line between L2 and R2');

plot3(mid1(1), mid1(2), mid1(3), 'k.','MarkerSize', 20, 'LineWidth', 2, ...
    'DisplayName','Midpoint1');
plot3(mid2(1), mid2(2), mid2(3), 'k.','MarkerSize', 20, 'LineWidth', 2, ...
    'DisplayName','Midpoint2');

disp('Frame 1 car coordinates wrt camera (x,y,z):'); disp(mid1);
disp('Frame 2 car coordinates wrt camera (x,y,z):'); disp(mid2);

 grid on; axis equal;
    xlabel('X [m]'); ylabel('Y [m]'); zlabel('Z [m]');
    title('Lights 3D position and car trajectory (coordinates wrt the camera');
    legend show;

```

```
if abs(angle_deg - 90) < threshold_deg
    disp('The directions are orthogonal (within the threshold), the car is moving forward.');

    plot3([mid1(1), mid2(1)], [mid1(2), mid2(2)], [mid1(3), mid2(3)], ...
    'k--', 'LineWidth', 2,'DisplayName','Car trajectory');
else
    disp('The directions are not orthogonal (within the threshold), the car in steering.');

    %%  CURVATURE ESTIMATION (using I.C.R. from rear lights)

    figure; hold on
    title('Car road plane position and trajectory (coordinates wrt the camera) / Steering case');

    dra=0.7; % Distance between rear axle and lights

    C1=[mid1(1)-car_length/2*sin(yaw0),mid1(3)+car_length/2*cos(yaw0)];
    C2=[mid2(1)-car_length/2*sin(yaw1),mid2(3)+car_length/2*cos(yaw1)];

    C1ra=[mid1(1)-dra*sin(yaw0),mid1(3)+dra*cos(yaw0)];
    C2ra=[mid2(1)-dra*sin(yaw1),mid2(3)+dra*cos(yaw1)];

    C1ral=[C1ra(1)-car_width/2*cos(yaw0),C1ra(2)-car_width/2*sin(yaw0)];
    C2ral=[C2ra(1)-car_width/2*cos(yaw1),C2ra(2)-car_width/2*sin(yaw1)];

    plot(C1(1),C1(2),'k.','MarkerSize',20,'DisplayName','Car center 1');
    plot(C2(1),C2(2),'k.','MarkerSize',20,'DisplayName','Car center 2');

    plot(C1ra(1),C1ra(2),'g.','MarkerSize',25,'DisplayName','Rear axle center 1');
    plot(C2ra(1),C2ra(2),'g.','MarkerSize',25,'DisplayName','Rear axle center 2');

    plot(C1ral(1),C1ral(2),'b.','MarkerSize',15,'DisplayName','C1l');
    plot(C2ral(1),C2ral(2),'b.','MarkerSize',15,'DisplayName','C2l');

    m1=(C1ra(2)-C1ral(2))/(C1ra(1)-C1ral(1)); % First line equation
    q1=-(m1*C1ra(1)-C1ra(2));

    m2=(C2ra(2)-C2ral(2))/(C2ra(1)-C2ral(1)); % Second line equation
    q2=-(m2*C2ra(1)-C2ra(2));

    x_icr=(q2-q1)/(m1-m2);
    z_icr=(m1*x_icr+q1);

    plot(pos_L1(1), pos_L1(3), 'ro','MarkerSize', 8, 'LineWidth', 2, 'DisplayName','L1');
    plot(pos_R1(1), pos_R1(3), 'ro','MarkerSize', 8, 'LineWidth', 2, 'DisplayName','R1');
    plot([pos_L1(1), pos_R1(1)], [pos_L1(3), pos_R1(3)], ...
    'r--', 'LineWidth', 1,'DisplayName','Virtual line between L1 and R1
```

```
    ');
232
233     plot(new_pos_L2(1), new_pos_L2(3), 'ro','MarkerSize', 8, 'LineWidth
    ', 2, 'DisplayName','L2');
234     plot(new_pos_R2(1), new_pos_R2(3), 'ro','MarkerSize', 8, 'LineWidth
    ', 2, 'DisplayName','R2');
235     plot([new_pos_L2(1), new_pos_R2(1)], [new_pos_L2(3), new_pos_R2(3)
    ], ...
236     'r--', 'LineWidth', 1,'DisplayName','Virtual line between L2 and R2
    ');
237
238     plot([x_icr, C1ra(1)], [z_icr, C1ra(2)], ...
239     'k--', 'LineWidth', 1,'DisplayName','Virtual radius 1');
240     plot([x_icr, C2ra(1)], [z_icr, C2ra(2)], ...
241     'k--', 'LineWidth', 1,'DisplayName','Virtual radius 2');
242
243     drawRotatedRectangle(C1, car_length, car_width, 3.14/2+yaw0);
244     drawRotatedRectangle(C2, car_length, car_width, 3.14/2+yaw1);
245
246     plot(x_icr, z_icr, 'mx','MarkerSize',12,'LineWidth',3,'DisplayName'
    ,'ICR');
247     xlabel('X [m]'); ylabel('Z [m]')
248
249     R1=sqrt((mid1(1)-x_icr)^2+(mid1(3)-z_icr)^2);
250     R2=sqrt((mid2(1)-x_icr)^2+(mid2(3)-z_icr)^2);
251     R=(R1+R2)/2;
252
253     fprintf('The radius of the curve is approximately: %.1f m \n',R);
254
255     theta = linspace(0, 2*3.14, 100);
256     x_circle=x_icr+R*cos(theta); z_circle=z_icr+R*sin(theta);
257
258     plot(x_circle, z_circle, 'm--', 'LineWidth',1, 'DisplayName','
    Trajectory circle');
259
260     grid on; axis equal; legend show
261 end
262
263 function drawRotatedRectangle(C1, L, W, theta)
264
265     % Coordinates
266     x = [-L/2,  L/2,  L/2, -L/2, -L/2];
267     y = [-W/2, -W/2,  W/2,  W/2, -W/2];
268
269     % Rotation matrix
270     R = [cos(theta), -sin(theta);
271          sin(theta),  cos(theta)];
272
273     % Rotation
274     rotated = R * [x; y];
275
276     %Translation
277     x_rot = rotated(1,:) + C1(1);
278     y_rot = rotated(2,:) + C1(2);
279
280     % Draw
281     plot(x_rot, y_rot, 'k-', 'LineWidth', 1,'DisplayName',"Vehicle's
    space occupancy");
```

```
    axis equal
    grid on
    hold on
end
```

Listing 5: Main trajectory analysis code for motion detection and 3D reconstruction

DETECTION_MOTION.py

```
clear; clc; close all;

%%  USER PARAMETERS

videoFile = 'ConstantCurve.mp4';   % Path to the video
threshold_deg = 2;          % Angular threshold for orthogonality check
    (degrees)

% Camera calibration matrix (example, replace with your own)
K = [1855  0    574;
       0 1850   965;
       0   0     1];

car_width=1.8;
car_length=4.1;
d_lights=1.294;   % distance between rear lights [m]

%%  LOAD VIDEO

v = VideoReader(videoFile);
nFrames = floor(v.Duration * v.FrameRate);

fprintf('Video loaded: %d frames available\n', nFrames);

%%  SELECT TWO FRAMES

frameIdx1=140; %1
frameIdx2=200; %100

frame1=read(v,frameIdx1);
frame2=read(v,frameIdx2);

%%  FRAME 1 (L1, R1)

figure;
imshow(frame1); hold on
title('L1 and R1 on frame 1')

L1 =[1160; 289; 1]; %[436; 236; 1];
R1 =[1649; 283; 1]; %[1452; 256; 1];

plot(L1(1), L1(2), 'g.', 'MarkerSize', 30, 'DisplayName', 'L1');
plot(R1(1), R1(2), 'c.', 'MarkerSize', 30, 'DisplayName', 'R1');
legend show;
hold off

%%  FRAME 2 (L2, R2)
```

```
47
48 figure;
49 imshow(frame2); hold on
50 title('L2 and R2 on frame 2')
51
52 L2=[1306; 313; 1]; %[540; 272; 1];
53 R2=[1620; 314; 1]; %[1353; 291; 1];
54
55 plot(L2(1), L2(2), 'g.', 'MarkerSize', 30, 'DisplayName', 'L2');
56 plot(R2(1), R2(2), 'c.', 'MarkerSize', 30, 'DisplayName', 'R2');
57 legend show;
58 hold off;
59
60 %%  COMPUTE IMAGE LINES
61
62 % Lines are computed as cross products of points
63 l_L1R1 = cross(L1, R1);
64 l_L2R2 = cross(L2, R2);
65
66 l_L1L2 = cross(L1, L2);
67 l_R1R2 = cross(R1, R2);
68
69 %%  COMPUTE VANISHING POINTS
70
71 vx = cross(l_L1R1, l_L2R2);
72 vy = cross(l_L1L2, l_R1R2);
73
74 % Normalize homogeneous coordinates
75 vx = vx / vx(3);
76 vy = vy / vy(3);
77
78 fprintf('Vanishing point vx = [%f %f %f]\n', vx);
79 fprintf('Vanishing point vy = [%f %f %f]\n', vy);
80
81 %%  ORTHOGONALITY CHECK USING K
82
83 % Convert vanishing points to direction vectors in camera frame
84 d_x=inv(K)*vx;
85 d_y=inv(K)*vy;
86
87 % Normalize directions
88 d_x=d_x/norm(d_x);
89 d_y=d_y/norm(d_y);
90
91 % Compute angle between directions
92 angle_rad = acos(dot(d_x, d_y));
93 angle_deg = rad2deg(angle_rad);
94
95 fprintf('Angle between directions = %.2f degrees\n', angle_deg);
96
97 %%  LINE AT INFINITY
98
99 linf = cross(vx, vy);
100 linf = linf / norm(linf(1:2));
101
102 fprintf('Line at infinity linf = [%f %f %f]\n', linf);
103
104 %%  PLANE AT INFINITY (BACKPROJECTION)
```

```
105
106 % Plane backprojection: pi = K^(-T) * linf
107 pi=K\linf;
108
109 % Normalize plane
110 pi=pi/norm(pi(1:3));
111
112 fprintf('Plane pi = [%f %f %f]\n', pi);
113
114 %% 3D SPACE POSITION
115
116 ray_L1=K\L1; ray_L1=ray_L1/norm(ray_L1);
117 ray_R1=K\R1; ray_R1=ray_R1/norm(ray_R1);
118
119 lambda1=d_lights/norm(ray_R1-ray_L1);
120 pos_L1=lambda1*ray_L1; pos_R1=lambda1*ray_R1;
121
122 ray_L2=K\L2; ray_L2=ray_L2/norm(ray_L2);
123 ray_R2=K\R2; ray_R2=ray_R2/norm(ray_R2);
124
125 lambda2=d_lights/norm(ray_R2-ray_L2);
126 pos_L2=lambda2*ray_L2; pos_R2=lambda2*ray_R2;
127
128 mid1=(pos_L1+pos_R1)/2;
129 mid2=(pos_L2+pos_R2)/2;
130
131 % Correction on the second frame
132
133 vec_L1R1=pos_R1([1,3])-pos_L1([1,3]);
134 %yaw0=atan2(vec_L1R1(2), vec_L1R1(1));
135 %rad2deg(yaw0)
136
137 yaw0=deg2rad(-45);
138 yaw1=yaw0+deg2rad(90-angle_deg);
139 rad2deg(yaw1)
140
141 %new_pos_L1=pos_L1;
142 pos_L1(1)=mid1(1)-d_lights/2*cos(yaw0);
143 pos_L1(3)=mid1(3)-d_lights/2*sin(yaw0);
144
145 pos_R1(1)=mid1(1)+d_lights/2*cos(yaw0);
146 pos_R1(3)=mid1(3)+d_lights/2*sin(yaw0);
147
148 new_pos_L2=pos_L2;
149 new_pos_L2(1)=mid2(1)-d_lights/2*cos(yaw1);
150 new_pos_L2(3)=mid2(3)-d_lights/2*sin(yaw1);
151
152 new_pos_R2=pos_R2;
153 new_pos_R2(1)=mid2(1)+d_lights/2*cos(yaw1);
154 new_pos_R2(3)=mid2(3)+d_lights/2*sin(yaw1);
155
156 %% PLOT
157
158 figure; hold on;
159 plot3(pos_L1(1), pos_L1(2), pos_L1(3), 'ro','MarkerSize', 8, 'LineWidth
    ', 2, ...
160     'DisplayName','L1');
161 plot3(pos_R1(1), pos_R1(2), pos_R1(3), 'ro','MarkerSize', 8, 'LineWidth
```

```
    ', 2, ...
      'DisplayName','R1');
plot3([pos_L1(1), pos_R1(1)], [pos_L1(2), pos_R1(2)], [pos_L1(3),
    pos_R1(3)], ...
     'r--', 'LineWidth', 1,'DisplayName','Virtual line between L1 and R1
    ');

plot3(new_pos_L2(1), new_pos_L2(2), new_pos_L2(3), 'ro','MarkerSize',
    8, 'LineWidth', 2, ...
      'DisplayName','L2');
plot3(new_pos_R2(1), new_pos_R2(2), new_pos_R2(3), 'ro','MarkerSize',
    8, 'LineWidth', 2, ...
      'DisplayName','R2');
plot3([new_pos_L2(1), new_pos_R2(1)], [new_pos_L2(2), new_pos_R2(2)], [
    new_pos_L2(3), new_pos_R2(3)], ...
     'r--', 'LineWidth', 1,'DisplayName','Virtual line between L2 and R2
    ');

plot3(mid1(1), mid1(2), mid1(3), 'k.','MarkerSize', 20, 'LineWidth', 2,
     ...
     'DisplayName','Midpoint1');
plot3(mid2(1), mid2(2), mid2(3), 'k.','MarkerSize', 20, 'LineWidth', 2,
     ...
     'DisplayName','Midpoint2');

disp('Frame 1 car coordinates wrt camera (x,y,z):'); disp(mid1);
disp('Frame 2 car coordinates wrt camera (x,y,z):'); disp(mid2);

 grid on; axis equal;
     xlabel('X [m]'); ylabel('Y [m]'); zlabel('Z [m]');
     title('Lights 3D position and car trajectory (coordinates wrt the
    camera');
     legend show;

if abs(angle_deg - 90) < threshold_deg
     disp('The directions are orthogonal (within the threshold), the car
     is moving forward.');

     plot3([mid1(1), mid2(1)], [mid1(2), mid2(2)], [mid1(3), mid2(3)],
    ...
     'k--', 'LineWidth', 2,'DisplayName','Car trajectory');
else
     disp('The directions are not orthogonal (within the threshold), the
     car in steering.');

     %%  CURVATURE ESTIMATION (using I.C.R. from rear lights)

     figure; hold on
     title('Car road plane position and trajectory (coordinates wrt the
    camera) / Steering case');

     dra=0.7; % Distance between rear axle and lights

     C1=[mid1(1)-car_length/2*sin(yaw0),mid1(3)+car_length/2*cos(yaw0)];
     C2=[mid2(1)-car_length/2*sin(yaw1),mid2(3)+car_length/2*cos(yaw1)];

     C1ra=[mid1(1)-dra*sin(yaw0),mid1(3)+dra*cos(yaw0)];
     C2ra=[mid2(1)-dra*sin(yaw1),mid2(3)+dra*cos(yaw1)];
```

```
206
207     C1ral=[C1ra(1)-car_width/2*cos(yaw0),C1ra(2)-car_width/2*sin(yaw0)
    ];
208     C2ral=[C2ra(1)-car_width/2*cos(yaw1),C2ra(2)-car_width/2*sin(yaw1)
    ];
209
210     plot(C1(1),C1(2),'k.','MarkerSize',20,'DisplayName','Car center 1')
    ;
211     plot(C2(1),C2(2),'k.','MarkerSize',20,'DisplayName','Car center 2')
    ;
212
213     plot(C1ra(1),C1ra(2),'g.','MarkerSize',25,'DisplayName','Rear axle
    center 1');
214     plot(C2ra(1),C2ra(2),'g.','MarkerSize',25,'DisplayName','Rear axle
    center 2');
215
216     plot(C1ral(1),C1ral(2),'b.','MarkerSize',15,'DisplayName','C1l');
217     plot(C2ral(1),C2ral(2),'b.','MarkerSize',15,'DisplayName','C2l');
218
219     m1=(C1ra(2)-C1ral(2))/(C1ra(1)-C1ral(1)); % First line equation
220     q1=-(m1*C1ra(1)-C1ra(2));
221
222     m2=(C2ra(2)-C2ral(2))/(C2ra(1)-C2ral(1)); % Second line equation
223     q2=-(m2*C2ra(1)-C2ra(2));
224
225     x_icr=(q2-q1)/(m1-m2);
226     z_icr=(m1*x_icr+q1);
227
228     plot(pos_L1(1), pos_L1(3), 'ro','MarkerSize', 8, 'LineWidth', 2, '
    DisplayName','L1');
229     plot(pos_R1(1), pos_R1(3), 'ro','MarkerSize', 8, 'LineWidth', 2, '
    DisplayName','R1');
230     plot([pos_L1(1), pos_R1(1)], [pos_L1(3), pos_R1(3)], ...
231     'r--', 'LineWidth', 1,'DisplayName','Virtual line between L1 and R1
    ');
232
233     plot(new_pos_L2(1), new_pos_L2(3), 'ro','MarkerSize', 8, 'LineWidth
    ', 2, 'DisplayName','L2');
234     plot(new_pos_R2(1), new_pos_R2(3), 'ro','MarkerSize', 8, 'LineWidth
    ', 2, 'DisplayName','R2');
235     plot([new_pos_L2(1), new_pos_R2(1)], [new_pos_L2(3), new_pos_R2(3)
    ], ...
236     'r--', 'LineWidth', 1,'DisplayName','Virtual line between L2 and R2
    ');
237
238     plot([x_icr, C1ra(1)], [z_icr, C1ra(2)], ...
239     'k--', 'LineWidth', 1,'DisplayName','Virtual radius 1');
240     plot([x_icr, C2ra(1)], [z_icr, C2ra(2)], ...
241     'k--', 'LineWidth', 1,'DisplayName','Virtual radius 2');
242
243     drawRotatedRectangle(C1, car_length, car_width, 3.14/2+yaw0);
244     drawRotatedRectangle(C2, car_length, car_width, 3.14/2+yaw1);
245
246     plot(x_icr, z_icr, 'mx','MarkerSize',12,'LineWidth',3,'DisplayName'
    ,'ICR');
247     xlabel('X [m]'); ylabel('Z [m]')
248
249     R1=sqrt((mid1(1)-x_icr)^2+(mid1(3)-z_icr)^2);
```

```
    R2=sqrt((mid2(1)-x_icr)^2+(mid2(3)-z_icr)^2);
    R=(R1+R2)/2;

    fprintf('The radius of the curve is approximately: %.1f m \n',R);

    theta = linspace(0, 2*3.14, 100);
    x_circle=x_icr+R*cos(theta); z_circle=z_icr+R*sin(theta);

    plot(x_circle, z_circle, 'm--', 'LineWidth',1, 'DisplayName','
   Trajectory circle');

    grid on; axis equal; legend show
end

function drawRotatedRectangle(C1, L, W, theta)

    % Coordinates
    x = [-L/2,  L/2,  L/2, -L/2, -L/2];
    y = [-W/2, -W/2,  W/2,  W/2, -W/2];

    % Rotation matrix
    R = [cos(theta), -sin(theta);
         sin(theta),  cos(theta)];

    % Rotation
    rotated = R * [x; y];

    %Translation
    x_rot = rotated(1,:) + C1(1);
    y_rot = rotated(2,:) + C1(2);

    % Draw
    plot(x_rot, y_rot, 'k-', 'LineWidth', 1,'DisplayName',"Vehicle's
   space occupancy");
    axis equal
    grid on
    hold on
end
```

Listing 6: Python implementation for motion detection and classification